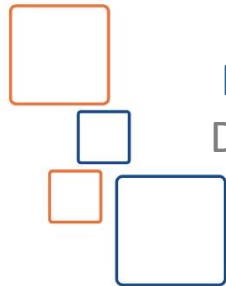


CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By
Dr. Eman Hesham, DBA.



Java™ Education
and Technology Services



Invest In Yourself,
Develop Your Career

A None-stop learner, Instructor, Coach, Mentor, Consultant ,SME, Researcher who is always self-motivated , Passionate to learn

- got DBA in modern teaching methodologies
- got her MSc in the data security field
- is Certified Application Security Engineer - CASE - EC-Council
- is Oracle Certified Web Component Developer
- is Certified Agile Scrum Master
- is Teaching Specialist in AI & DS -EPITA
- got ITI- 9-month diploma, intake 26, Java Department
- is Subject Matter Expert (SME) for online course creation & production at Mahar-Tech
 - Java Course with 28,000 + online learner
- is Training Consultant for training and development for different companies
- 18+ years of experience of teaching and development
 - ITI students(9month, 3 month , AI program and Egyptian universities)
 - Africa
 - Ministry of defense and Ministry of finance
 - Multinational companies.
- Coding for kids and STEM Coach for many years
- Applies Agile in training & online tech technologies.
- participated in the organization and preparation of MCIT, ITI and JETS events(JETS Launching Seminar and Workshop, MDW)



Dr. Eman Hesham , DBA.
MCIT- ITI - JETS
Executive Manager
ehesham@iti.gov.eg
Hesham.eman@gmail.com

Online Java development Course

► <https://maharatech.gov.eg/course/view.php?id=2052>

CH01 - The History and Evolution of Java

CH02 - An Overview of Java

CH03 - Datatypes ,Variables and Arrays

CH04 - Operators, Control Statements and String Handling

CH05 - Packages and Interfaces

CH06 - Wrapper Classes, Enumeration, Autoboxing and Annotations

CH07 - Exception Handling

CH08 - Generics and Lambda Expressions

CH09 - Java Collections

CH10 - Java Stream API

CH11 - Threads



Course Duration and Evaluation

- ▶ Duration: 42 hours
 - ▶ 7 Lectures
 - ▶ 7 Labs
- ▶ Evaluation Criteria:
 - ▶ 40% on Lectures/labs activities, Maharatech course and assignments
 - ▶ 60% on written exam after 7 days of the last lectures.



Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

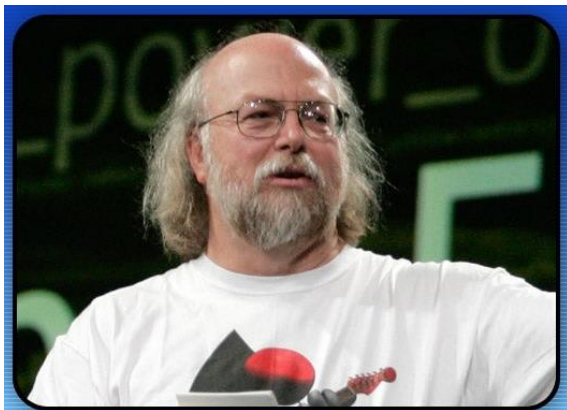
Lesson 1

An Overview of Java's History and Evolution



The History and Evolution of Java

- ▶ By the end of the 1980s and the early 1990s, object-oriented programming using C++ took hold.
- ▶ Java was conceived by **James Gosling, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan** at Sun Microsystems, Inc. in 1991.
- ▶ It took 18 months to develop the first working version.
- ▶ This language was initially called “Oak,” but was renamed “Java” in 1995



The History and Evolution of Java

- ▶ The primary motivation was the need for a **platform-independent** (that is, architecture-neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
- ▶ The trouble with C and C++ (and most other languages) is that they are designed to be compiled for a specific target
- ▶ Java derives much of its character from C and C++.
- ▶ Java also embodies changes in the way that people approach the writing of programs.
- ▶ For example, Java enhanced and refined the object-oriented paradigm used by C++, added integrated support for multithreading, and provided a library that simplified Internet access.

The History and Evolution of Java

How Java Impacted the Internet?

- ▶ The Internet helped pushing Java to the forefront of programming, and Java, in turn, had a deep effect on the Internet.
- ▶ In addition to simplifying web programming in general, Java innovated a new type of networked program called the applet that changed the way the online world thought about content.
- ▶ Java also addressed some of the trickiest issues associated with the Internet:
 - ▶ portability and security. Let's give some brief information about each of these:
 - ▶ Java Applets
 - ▶ Security
 - ▶ Portability

The History and Evolution of Java

How Java Impacted the Internet?

Java Applets

- ▶ At the time of Java's creation, one of its most exciting features was the applet.
- ▶ An applet is a special kind of Java program that is designed to be transmitted over the Internet and automatically executed inside a Java-compatible web browser.
- ▶ They were typically used to display data provided by the server, handle user input, or provide simple functions, such as a loan calculator, that execute locally, rather than on the server. In essence, the applet allowed some functionality to be moved from the server to the client.
- ▶ In the early days of Java, applets were a crucial part of Java programming.
- ▶ Starting with JDK 9, applets are being phased out and deprecated, with other mechanisms supplying an alternative way of delivering dynamic, active programs via the Web

The History and Evolution of Java

How Java Impacted the Internet

Security

- ▶ As you are likely aware, every time you download a “normal” program, you are taking a risk, because the code you are downloading might contain a virus, Trojan horse, or other harmful code.
- ▶ Java achieved this protection by enabling you to confine an application to the Java execution environment and prevent it from accessing other parts of the computer.
- ▶ The ability to download programs with a degree of confidence that no harm will be done may have been one of the the most innovative aspect of Java at that time.

The History and Evolution of Java

How Java Impacted the Internet

Portability

- ▶ Portability is a major aspect of the Internet because there are many different types of computers and operating systems connected to it.
- ▶ If a Java program were to be run on virtually any computer connected to the Internet, there needed to be some way to enable that program to execute on different systems.
- ▶ The same code must work on all computers. Therefore, some means of generating portable executable code was needed.
- ▶ The same mechanism that helps ensure security also helps create portability.

Java's Magic: The Bytecode

- ▶ The key that allows Java to solve both the security and the portability problems just described is that the output of a Java compiler is not executable code. Rather, it is bytecode.
- ▶ Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).
- ▶ In essence, the original JVM was designed as an interpreter for bytecode.
- ▶ However, the fact that a Java program is executed by the JVM helps solve the major problems associated with web-based programs.

Java's Magic: The Bytecode

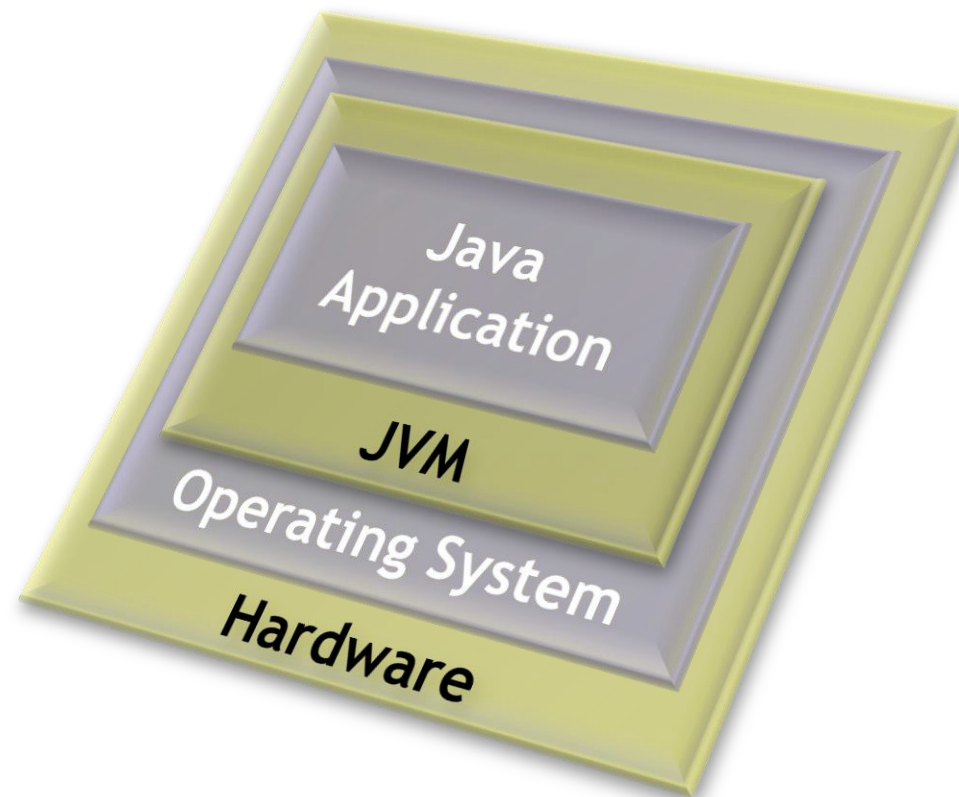
- ▶ Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
- ▶ Once the run-time package exists for a given system, any Java program can run on it. Remember, although the details of the JVM will differ from platform to platform, all understand the same Java bytecode.
- ▶ The fact that a Java program is executed by the JVM also helps to make it secure. Because the JVM is in control, it manages program execution.

Java's Magic: The Bytecode

- ▶ It is possible for the JVM to create a restricted execution environment, called the **sandbox**, that contains the program, preventing unrestricted access to the machine.
- ▶ Although Java was designed as an interpreted language, there is nothing about Java that prevents on-the-fly compilation of bytecode into native code in order to boost performance.

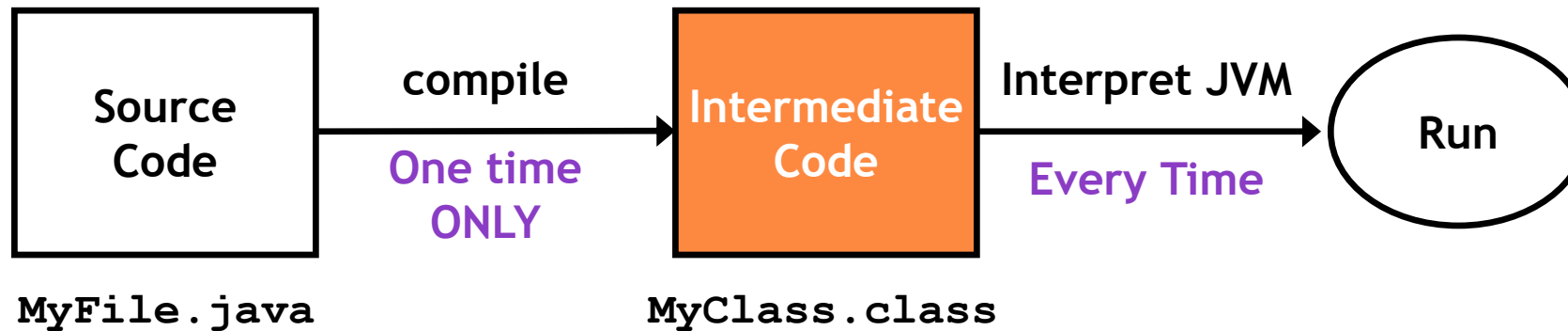
Java's Magic:

- ▶ Machine and Platform Independent (Architecture Neutral)



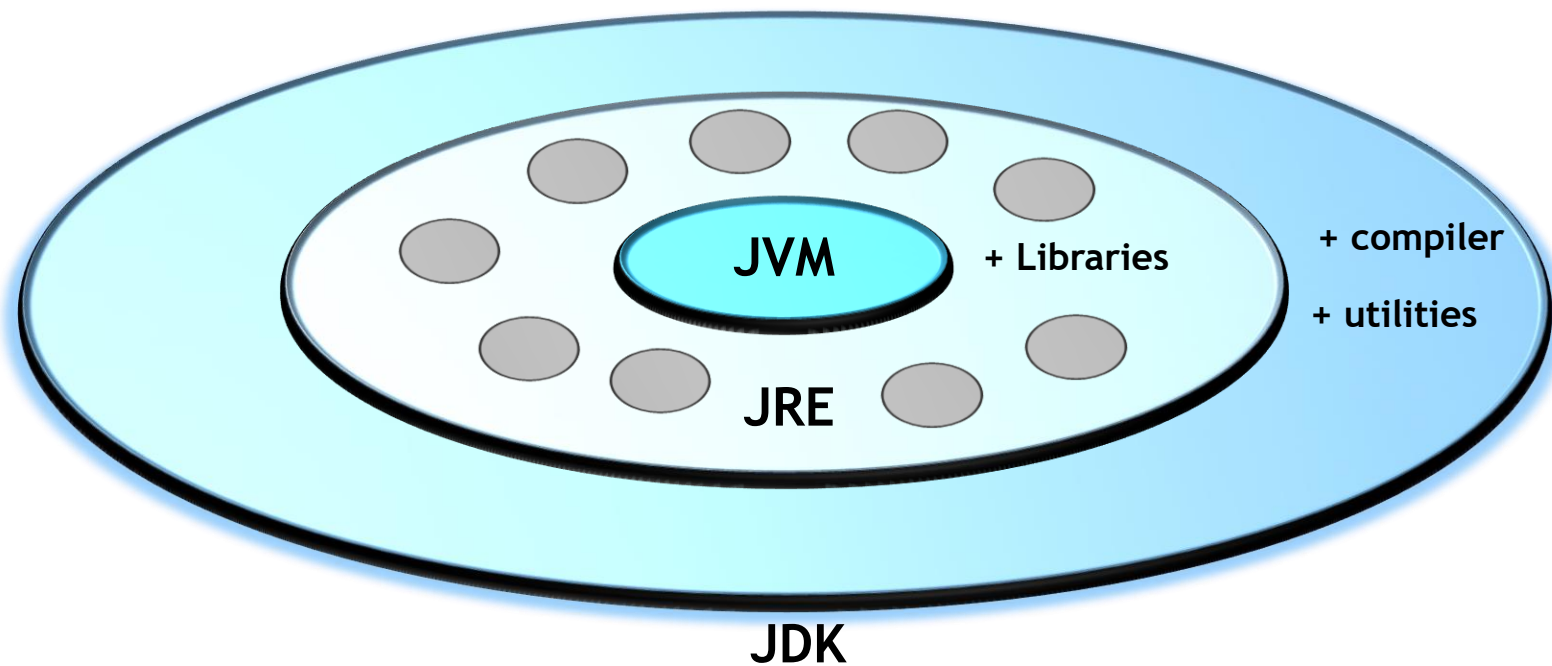
Java's Magic:

- ▶ Java is both, compiled and interpreted

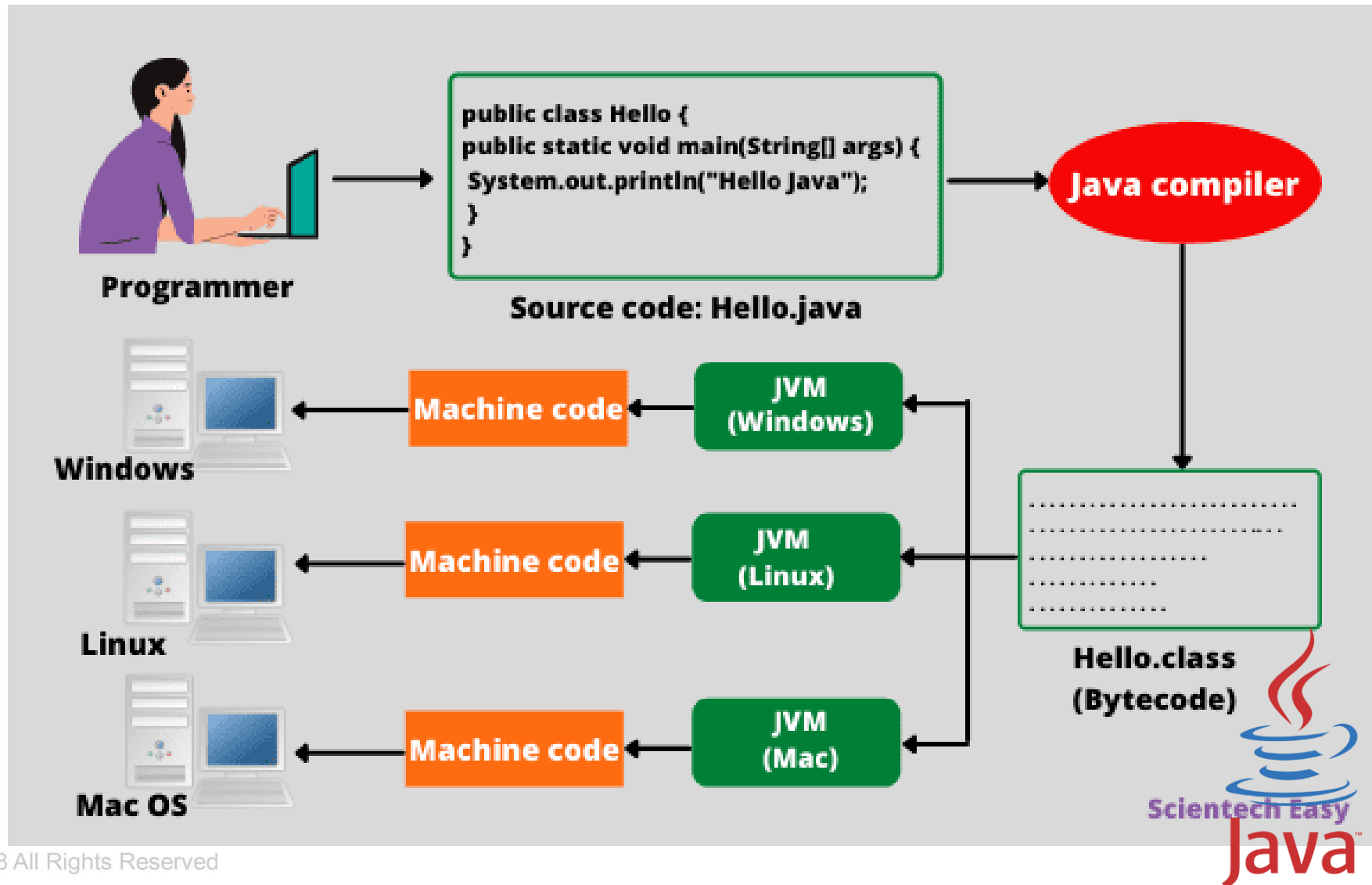


Java's Magic:

- ▶ Java depends on dynamic linking of libraries



Java's Magic:



The Java Buzzwords

- ▶ No discussion of Java's history is complete without a look at the Java buzzwords.
- ▶ The key considerations were summed up by the Java team in the following list of buzzwords:

Simple	Secure	Portable	Object-oriented
Robust	Multithreaded	Architecture-neutral	Interpreted
High performance	Distributed	Dynamic	

The Java Buzzwords

▶ Simple

- ▶ Java was designed to be easy for the professional programmer to learn and use effectively.
- ▶ If you already understand the basic concepts of object oriented programming, learning Java will be even easier.
- ▶ If you are an experienced C++ programmer, moving to Java will require very little effort.

▶ Object-Oriented

- ▶ The object model in Java is simple and easy to extend, while primitive types, such as integers, are kept as high-performance non objects

Simple	Secure	Portable	Object-oriented
Robust	Multithreaded	Architecture-neutral	Interpreted
High performance	Distributed	Dynamic	

The Java Buzzwords

▶ Robust

- ▶ Because Java is a strictly typed language, it checks your code at compile time. However, it also checks your code at run time. To better understand how Java is robust, consider two of the main reasons for program failure: memory management mistakes and mishandled exceptional conditions (that is, run-time errors).

▶ Multithreaded

- ▶ Java supports multithreaded programming, which allows you to write programs that do many things simultaneously.
- ▶ The Java run-time system comes with an elegant yet sophisticated solution for multiprocess synchronization that enables you to construct smoothly running interactive systems.

Simple	Secure	Portable	Object-oriented
Robust	Multithreaded	Architecture-neutral	Interpreted
High performance	Distributed	Dynamic	

The Java Buzzwords

▶ Architecture-neutral

- ▶ Their goal was “*write once; run anywhere.*” To a great extent, this goal was accomplished.

▶ Interpreted and High Performance

- ▶ Java enables the creation of cross-platform programs by compiling into an intermediate representation called Java *bytecode*.
- ▶ This code can be executed on any system that implements the Java Virtual Machine.
- ▶ The Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a just-in-time compiler

Simple	Secure	Portable	Object-oriented
Robust	Multithreaded	Architecture-neutral	Interpreted
High performance	Distributed	Dynamic	

The Java Buzzwords

▶ Distributed

- ▶ Java is designed for the distributed environment of the Internet because it handles TCP/IP protocols.
- ▶ Java also supports Remote Method Invocation (RMI), this feature enables a program to invoke methods across a network.

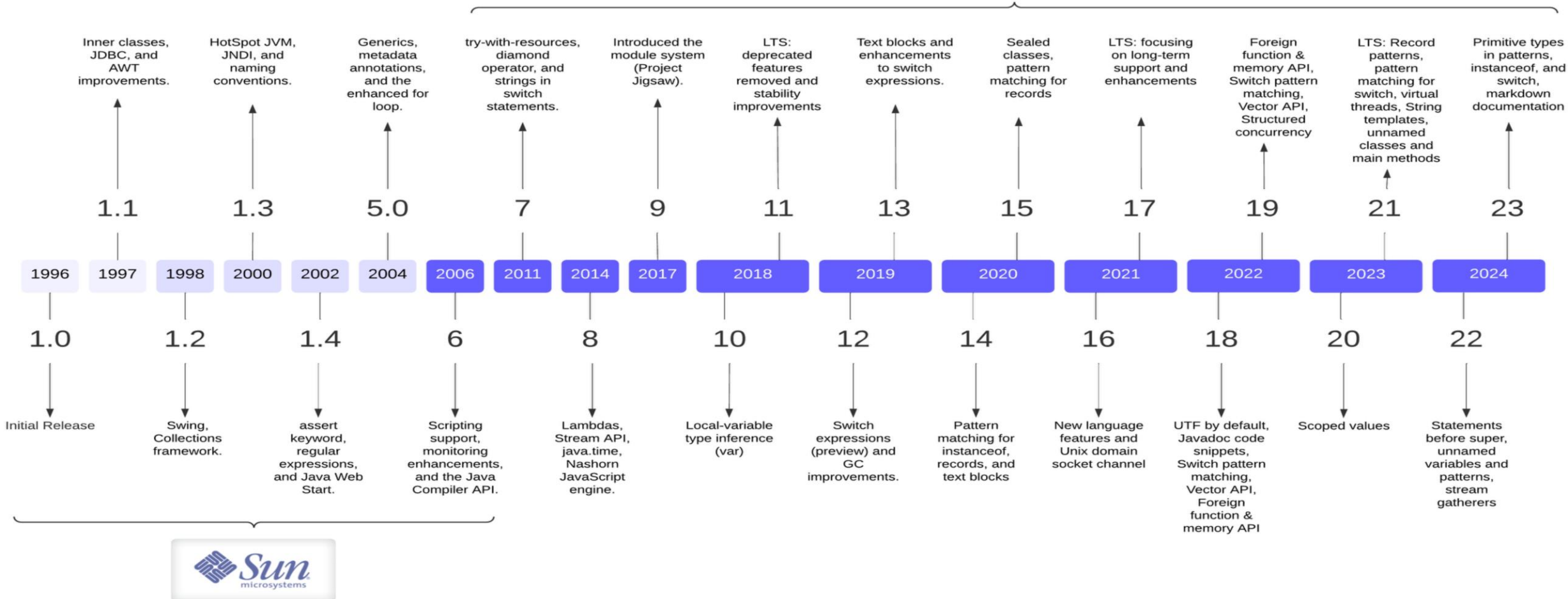
▶ Dynamic

- ▶ Java programs carry with them substantial amounts of run-time type information that is used to verify and resolve access to objects at run time. This makes it possible to **dynamically link** code in a safe and convenient manner. This is crucial to the robustness of the Java environment, in which small fragments of bytecode may be dynamically updated on a running system.

Simple	Secure	Portable	Object-oriented
Robust	Multithreaded	Architecture-neutral	Interpreted
High performance	Distributed	Dynamic	

The Evolution of Java

ORACLE



Agenda

- A. The History and Evolution of Java
- B. An Overview of Java**
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 2

An Overview of Java



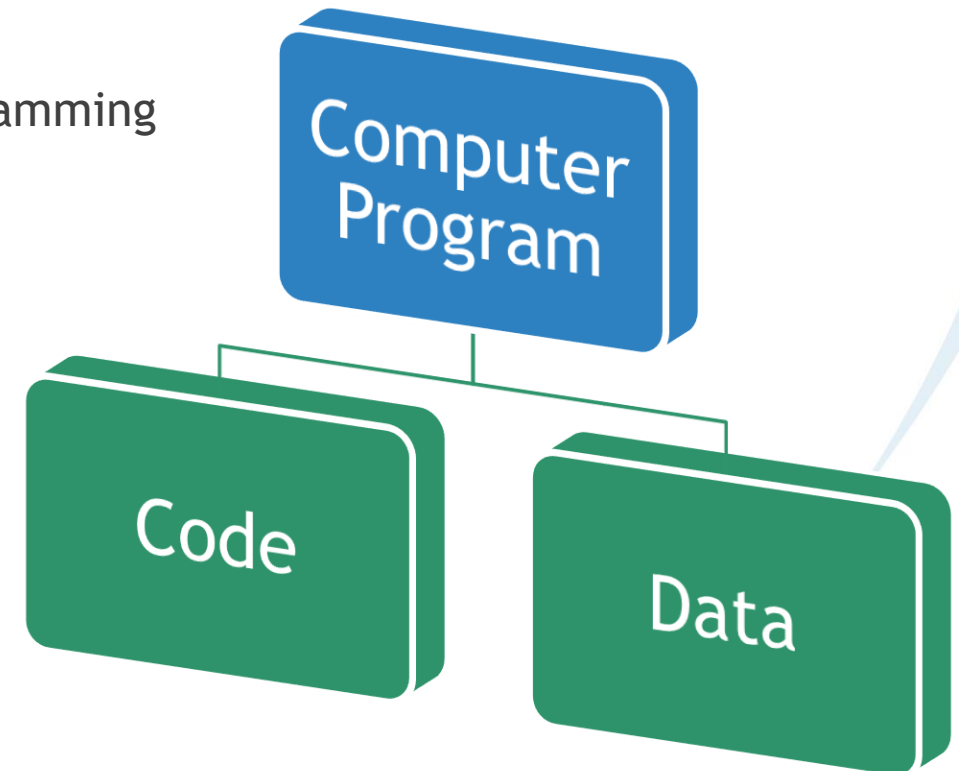
An Overview of Java

▶ Object-Oriented Programming

- ▶ Object-oriented programming (OOP) is at the core of Java.
- ▶ OOP is so integral to Java that it is best to understand its basic principles before you begin writing even simple Java programs.

Computer Program Components

- ▶ These are the two paradigms that govern how a program is constructed:
 - ▶ The first way is called the **process-oriented** model.
 - ▶ The second approach, called **object-oriented** programming



The Process-Oriented model

- ▶ The process oriented model can be thought of as code acting on data.
- ▶ Procedural languages such as **C** employ this model to considerable success.
- ▶ However, the problem with this approach appear as programs grow larger and more complex.

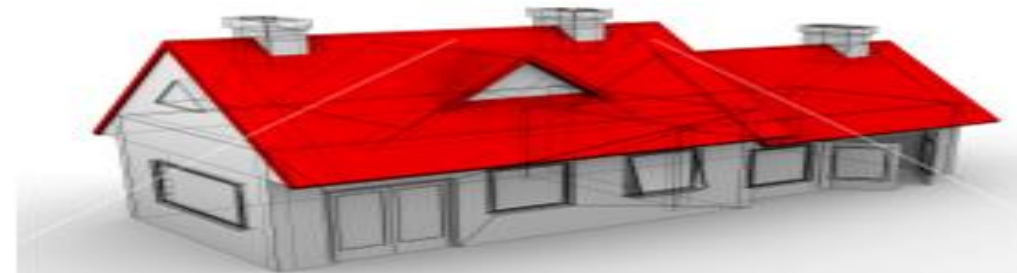
The Object-Oriented Programming

- ▶ Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.
- ▶ An object-oriented program can be characterized as data controlling access to code.
- ▶ We will be discussing the main characteristics of Object Oriented

class

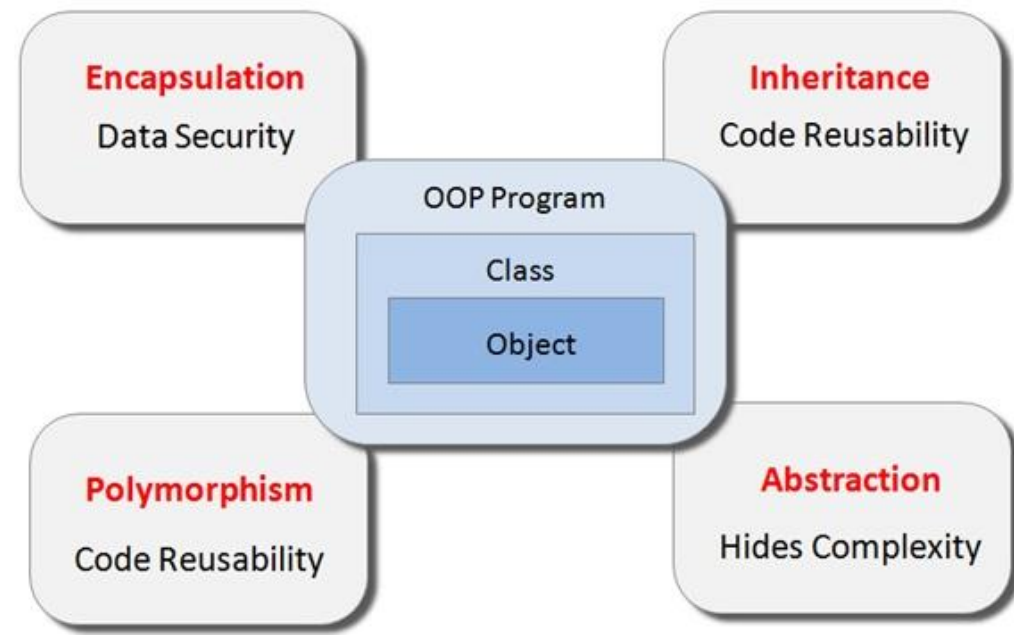


objects



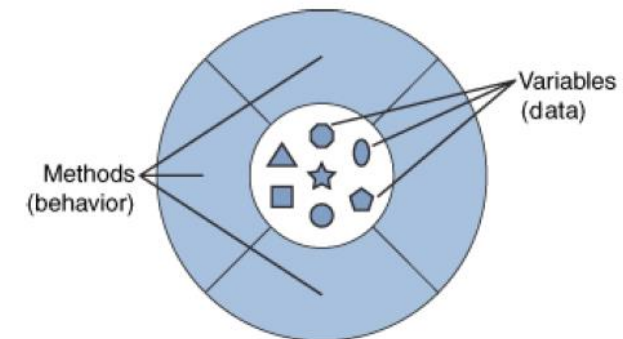
The OOP Principles

- ▶ All object-oriented programming languages provide mechanisms that help you implement the object-oriented model.
- ▶ They are **Abstraction**, **encapsulation**, **inheritance**, and **polymorphism**.
- ▶ In the following slides we will be discussing these concepts briefly.



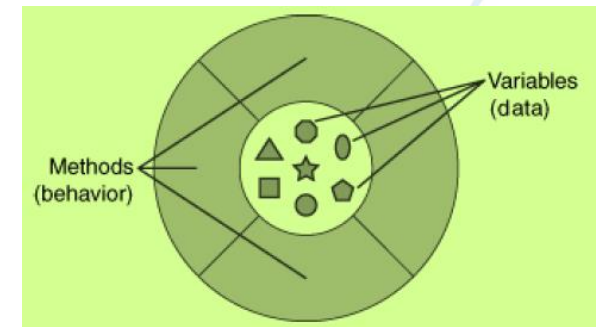
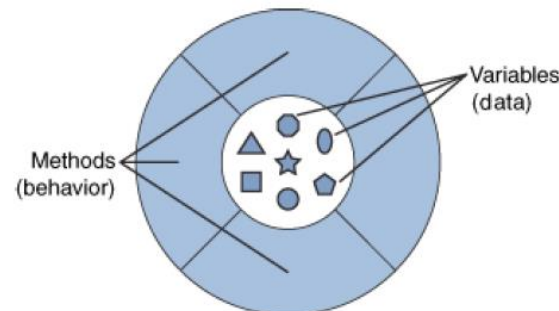
Abstraction

- ▶ An essential element of object-oriented programming is abstraction.
- ▶ Humans manage complexity through abstraction.
- ▶ For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior.
- ▶ A powerful way to manage abstraction is through the use of hierarchical classifications.
- ▶ This allows you to layer the semantics of complex systems, breaking them into more manageable pieces.



Abstraction

- ▶ From the outside, the car is a single object.
- ▶ Once inside, you see that the car consists of several subsystems: steering, brakes, sound system, seat belts, heating, cellular phone, and so on. In turn, each of these subsystems is made up of more specialized units.
- ▶ Hierarchical abstractions of complex systems can also be applied to computer programs.
- ▶ The data from a traditional process-oriented program can be transformed by abstraction into its component objects.
- ▶ A sequence of process steps can become a collection of messages between these objects.



Abstraction

- ▶ Thus, each of these objects describes its own **unique** behavior.
- ▶ You can treat these objects as **concrete** entities that respond to messages telling them to do something.
- ▶ As you will see, object-oriented programming is a powerful and natural paradigm for creating programs that survive the changes accompanying the life cycle of any major software project, including conception, growth, and aging.

Encapsulation

- ▶ Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.
- ▶ In Java, the basis of encapsulation is the **class**.
- ▶ A **class** defines the structure and behavior (data and code) that will be shared by a set of objects.
- ▶ Each object of a given class contains the structure and behavior defined by the class.
- ▶ Thus, a class is a logical construct; an object has physical reality.

Encapsulation

- ▶ When you create a class, you will specify the code and data that constitute that class.
- ▶ The data defined by the class are referred to as member variables or instance variables.
- ▶ The code that operates on that data is referred to as member methods or just methods.

Encapsulation- A Class Representation



Public instance
variables (not
recommended)



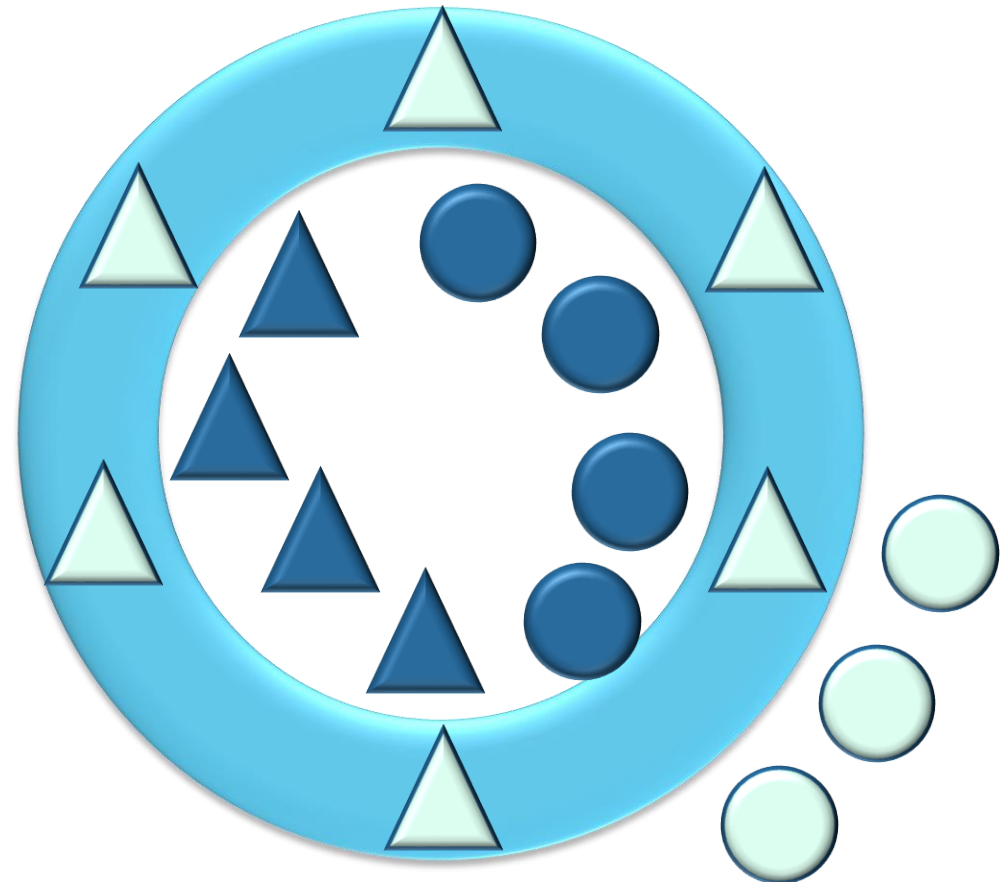
Public methods



Private methods

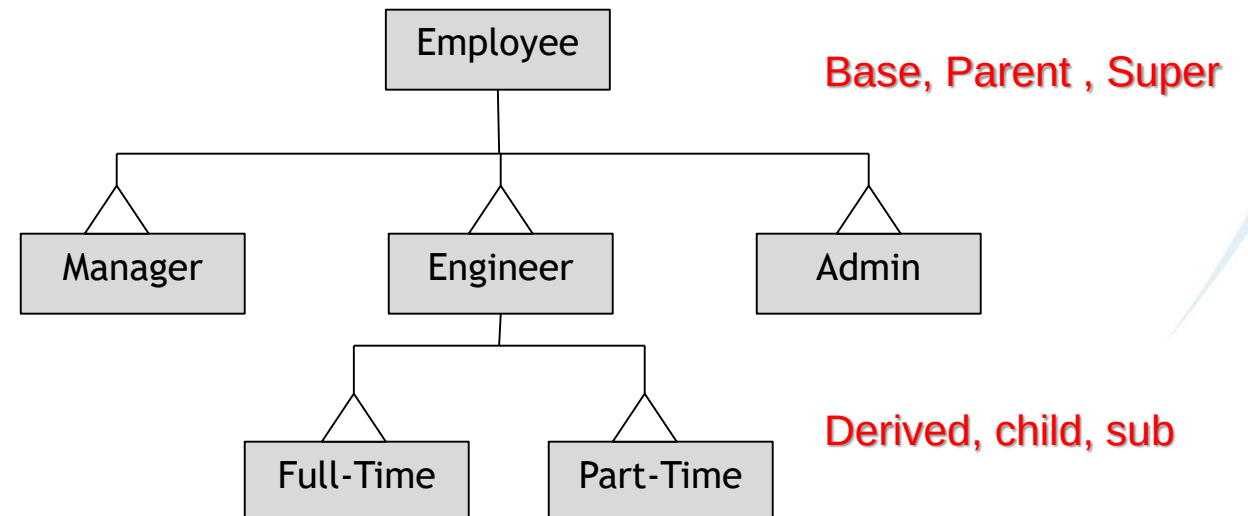
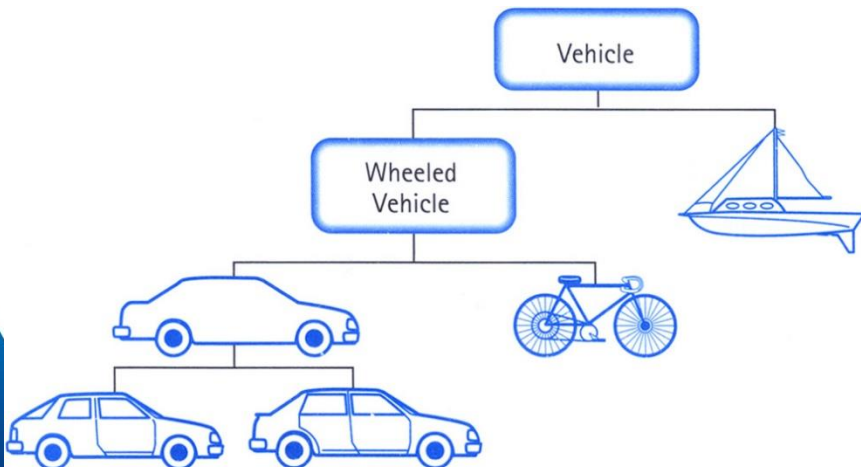


Private instance
variables



Inheritance

- ▶ Inheritance is the process by which one object acquires the properties of another object.
- ▶ This is important because it supports the concept of hierarchical classification.
- ▶ Without the use of hierarchies, each object would need to define all of its characteristics explicitly.
- ▶ By use of inheritance, an object need only define those qualities that make it unique within its class. It can inherit its general attributes from its parent.

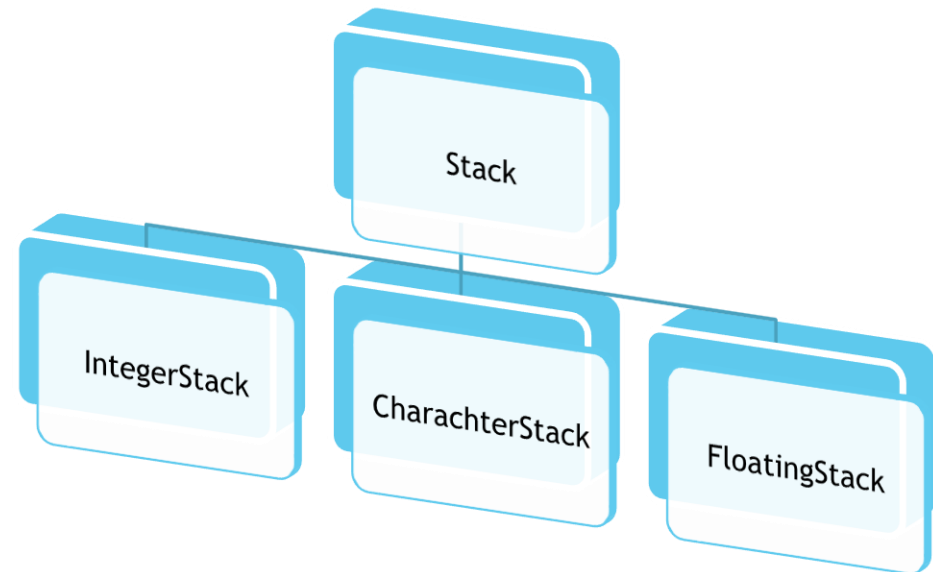


Polymorphism

- ▶ Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.
- ▶ More generally, the concept of polymorphism is often expressed by the phrase “one interface, multiple methods.” This means that it is possible to design a generic interface to a group of related activities.
- ▶ Consider a stack (which is a last-in, first-out list). You might have a program that requires three types of stacks. One stack is used for integer values, one for floating-point values, and one for characters.

Polymorphism

- ▶ It is the compiler's job to select the specific action (that is, method) as it applies to each situation.
- ▶ You, the programmer, do not need to make this selection manually. You need only remember and utilize the general interface.
- ▶ The specific action would be determined depending on the calling object



First Java Program

Example.java

```
/*  
This is a simple Java program.  
Call this file "Example.java".  
*/  
  
public class Example {  
    // Your program begins with a call to main().  
    public static void main(String args[]) {  
        System.out.println("This is a simple Java program.");  
    }  
}
```

First Java Program

- ▶ For this example, the name of the source file must be Example.java.
- ▶ In Java, a source file is officially called a compilation unit.
- ▶ It is a text file that contains (among other things) one or more class definitions.
- ▶ The Java compiler requires that a source file use the .java filename extension.

Compiling the Program

- ▶ To compile the Example program, execute the compiler, javac, specifying the name of the source file on the command line, as shown here:

```
C:\>javac Example.java
```

- ▶ The javac compiler creates a file called Example.class that contains the bytecode version of the program.
- ▶ As discussed earlier, the Java bytecode is the intermediate representation of your program that contains instructions the Java Virtual Machine will execute.

Running the Program

- ▶ To actually run the program, you must use the Java application launcher called java. To do so, pass the class name Example as a command-line argument, as shown here:

```
C:\>java Example
```

- ▶ When the program is run, the following output is displayed:

```
C:\>This is a simple Java program.
```

First Sample Program Examined

- ▶ The program begins with the following lines:

```
/*
```

```
*This is a simple Java program.
```

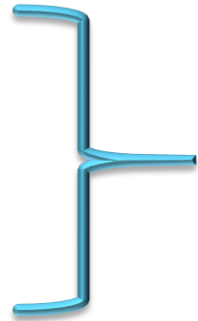
```
*Call this file "Example.java".
```

```
*/
```

- ▶ The Above lines are multiline comments that are considered as remarks for the developer, explaining the operation of the program to anyone who is reading its source code.

Java three Styles of Comments

```
/*  
Comment  
*/
```



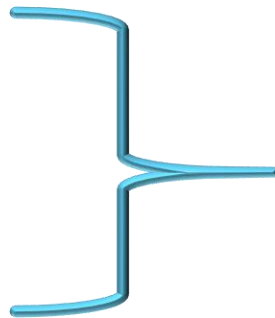
Multiline Comment

```
// Comment
```



Single Line Comment

```
/**  
Comment  
*/
```



**Comments processed by Javadoc
tool**

First Sample Program Examined

- ▶ The next line of code in the program is shown here:

```
class Example {
```

- ▶ This line uses the keyword ***class*** to declare that a new class is being defined.
- ▶ **Example** is an identifier that is the name of the class.
- ▶ The entire class definition, including all of its members, will be between the opening curly brace ({) and the closing curly brace (}).

First Sample Program Examined

- ▶ The next line of code is shown here:

```
public static void main(String args[]) {
```

- ▶ As a general rule, a Java program begins execution by calling `main( )`.
- ▶ The `public` keyword is an access modifier, which allows the programmer to control the visibility of class members.
- ▶ When a class member is preceded by `public`, then that member may be accessed by code outside the class in which it is declared.
- ▶ In this case, `main( )` must be declared as `public`, since it must be called by code outside of its class when the program is started.

First Sample Program Examined

- ▶ The keyword **static** allows **main()** to be called without having to instantiate a particular instance of the class.
- ▶ This is necessary since **main()** is called by the Java Virtual Machine before any objects are made.
- ▶ The keyword **void** simply tells the compiler that **main()** does not return a value.
- ▶ **String args[]** declares a parameter named args, which is an array of instances of the class **String**.
- ▶ In this case, args receives any command-line arguments present when the program is executed.

First Sample Program Examined

- ▶ The next line of code is shown here. Notice that it occurs inside **main()**.

```
System.out.println("This is a simple Java program.");
```

- ▶ Output is actually accomplished by the built-in **println()** method.
- ▶ **System** is a predefined class that provides access to the system, and **out** is the output stream that is connected to the console.

Second Short Program

```
public class Example2 {  
    public static void main(String args[]) {  
        int num; // this declares a variable called num  
        num = 100; // this assigns num the value 100  
        System.out.println("This is num: " + num);  
        num = num * 2;  
        System.out.print("The value of num * 2 is ");  
        System.out.println(num);  
    }  
}
```

type var-name;

*assigns to num the value
100
the plus sign causes
the value of num to
be appended to the
string that precedes
it*

Two Control Statements- if statement

- ▶ The Java **if statement** works much like the **if statement** in any other language.
- ▶ It determines the flow of execution based on whether some condition is true or false. Its simplest form is shown here:

- ▶ *if(condition) statement;*

Where *condition* is a boolean expression that evaluates either *true* or *false*

```
if(num < 100)
```

```
System.out.println("x is less than y");
```

- ▶ if num contains a value that is less than 100, the conditional expression is true, and println() will execute.

If Statement Example

```
//Demonstrate the if. Call this file "IfSample.java".//
```

```
class IfSample {  
    public static void main(String args[]) {  
        int x, y;  
        x = 10; y = 20;  
        if(x < y) System.out.println("x is less than y");  
        x = x * 2;  
        if(x == y) System.out.println("x now equal to y");  
        x = x * 2;  
        if(x > y) System.out.println("x now greater than y");  
        // this won't display anything  
        if(x == y) System.out.println("you won't see this");  
    }  
}
```

Two Control Statements- the for Loop

- ▶ Loop statements are an important part of nearly any programming language because they provide a way to repeatedly execute some task.
- ▶ The simplest form of the for loop is shown here:

***for**(initialization; condition; iteration) statement;*

- ▶ The initialization portion of the loop sets a loop control variable to an initial value.
- ▶ The condition is a Boolean expression that tests the loop control variable.
- ▶ If the outcome of that test is true, statement executes and the for loop continues to iterate.
- ▶ If it is false, the loop terminates.
- ▶ The iteration expression determines how the loop control variable is changed each time the loop iterates.

For Loop Example

```
/*  
Demonstrate the for loop.  
Call this file "ForTest.java".  
*/  
class ForTest {  
    public static void main(String args[]) {  
        int x;  
        for(x = 0; x<10; x = x+1)  
            System.out.println("This is x: " + x);  
    }  
}
```

The Output:

This is x: 0
This is x: 1
This is x: 2
This is x: 3
This is x: 4
This is x: 5
This is x: 6
This is x: 7
This is x: 8
This is x: 9

Using Blocks of Code

- ▶ Java allows two or more statements to be grouped into *blocks of code*, also called *code blocks*.
- ▶ This is done by enclosing the statements between opening and closing curly braces.
- ▶ Once a block of code has been created, it becomes a logical unit that can be used any place that a single statement can.
- ▶ For example, a block can be a target for Java's if and for statements.

Block of Code Example

```
/*  
Demonstrate a block of code.  
Call this file "BlockTest.java"  
*/  
class BlockTest {  
    public static void main(String args[]) {  
        int x, y;  
        y = 20;  
        // the target of this loop is a block  
        for(x = 0; x<10; x++) {  
            System.out.println("This is x: " + x);  
            System.out.println("This is y: " + y);  
            y = y - 2;  
        }  
    }  
}
```

This is x: 0
This is y: 20
This is x: 1
This is y: 18
This is x: 2
This is y: 16
This is x: 3
This is y: 14
This is x: 4
This is y: 12
This is x: 5
This is y: 10
This is x: 6
This is y: 8
This is x: 7
This is y: 6
This is x: 8
This is y: 4
This is x: 9
This is y: 2

Lexical Issues- Atomic Elements of Java

- ▶ Java programs are a collection of whitespace, identifiers, literals, comments, operators, separators, and keywords.
- ▶ **Whitespace**
 - ▶ Java is a free-form language. This means that you do not need to follow any special indentation rules.

Lexical Issues- Atomic Elements of Java

► Identifier

- Identifiers are used to name things, such as classes, variables, and methods. An identifier may be any descriptive sequence of uppercase and lowercase letters, numbers, or the underscore and dollar-sign characters.
- They must not begin with a number, so as not to be confused with a numeric literal. Again, Java is case-sensitive, so `VALUE` is a different identifier than `Value`.

NOTE: Beginning with JDK 9, the underscore cannot be used by itself as an identifier.

Valid identifiers	AvgTemp	count	a4	\$test	this_is_ok
Invalid identifiers	2count	high-temp	Not/ok	new Value	

Lexical Issues- Atomic Elements of Java

► Literals

- A constant value in Java is created by using a literal representation of it. For example, here are some literals:

100	98.6	'X', '\u03c0'	“This is a test”
integral literal	floating-point value	Character Literal	String Literal

Lexical Issues- Atomic Elements of Java

▶ Separators

- ▶ In Java, there are a few characters that are used as separators. The most commonly used separator in Java is the semicolon. As you have seen, it is often used to terminate statements.
- ▶ The separators are shown in the following table:

Lexical Issues- Atomic Elements of Java

Symbol	Name	Purpose
()	Parentheses	Used to contain lists of parameters in method definition and invocation
{ }	Braces	Used to contain the value of automatically initialized arrays. Also used to define a block of code, for classes, methods and local scopes
[]	Brackets	Used to declare array types. Also used when dereferencing array values
;	Semicolon	Terminates statements

Lexical Issues- Atomic Elements of Java

Symbol	Name	Purpose
,	Comma	Separates consecutive identifiers in a variable declaration. Also used to chain statements together inside a for statement
.	Period	Used to separate package names from subpackages and classes. Also used to separate a variable or method from a reference variable
::	Colons	Used to create a method or constructor reference
...	Ellipses	Indicates a variable-arity parameter
@	Ampersand	Begins an annotation

Lexical Issues- Atomic Elements of Java

▶ The Java Keywords

- ▶ There are 61 keywords currently defined in the Java language.
- ▶ These keywords, combined with the syntax of the operators and separators, form the foundation of the Java language.
- ▶ In general, these keywords cannot be used as identifiers, meaning that they cannot be used as names for a variable, class, or method.
- ▶ The exceptions to this rule are the new context-sensitive keywords added by JDK 9 to support modules.
- ▶ The keywords **const** and **goto** are reserved but not used.
- ▶ In addition to the keywords, Java reserves the following: **true**, **false**, and **null**.

Lexical Issues- Atomic Elements of Java

► The Java Keywords

abstract	assert	boolean	break	byte	case	catch	char
class	const	continue	default	do	double	else	enum
exports	extends	final	finally	float	for	goto	if
implements	import	instanceof	int	interface	long	module	native
new	open	opens	package	private	protected	provides	public
requires	return	short	static	strictfp	super	switch	synchronized
this	throw	throws	to	transient	transitive	try	uses
void	volatile	while	with				



Lab Exercises

Lab Exercises

- ▶ Create simple application that performs the following actions:
 - ▶ Display a message to the command prompt.

```
C:\>java Example1  
C:\> Hello Java
```

Lab Exercises

- ▶ Create simple application that performs the following actions:
 - ▶ Receives an input (as main arguments) and checks for its value and prints it back.

```
C:\>java Example2 CoreJava  
C:\> true value CoreJava
```

Lab Exercises

- ▶ Create simple application that performs the following actions:
 - ▶ Receives two inputs (as main arguments) a number and a string and prints the string on different lines according to the given number.

```
C:\>java Example3 5 CoreJava  
C:\> CoreJava  
CoreJava  
CoreJava  
CoreJava  
CoreJava
```